Bridge Design pattern

Intent:

- Decouple an abstraction from its implementation so that the two can vary independently.
- Publish interface in an inheritance hierarchy, and bury implementation in its own inheritance hierarchy.
- Beyond encapsulation, to insulation

The Bridge Pattern is used to separate out the interface from its implementation. Doing this gives the flexibility so that both can vary independently.

Example:

The Bridge pattern decouples an abstraction from its implementation, so that the two can vary independently. A household switch controlling lights, ceiling fans, etc. is an example of the Bridge. The purpose of the switch is to turn a device on or off. The actual switch can be implemented as a pull chain, simple two position switch, or a variety of dimmer switches.

Here is the sample code for Switch:

Switch.java

```
package structural.bridge;
/**
* Just two methods. on and off.
*/
public interface Switch {
        // Two positions of switch.
        public void switchOn();
        public void switchOff();
}// End of interface
```

This switch can be implemented by various devices in house, as Fan, Light Bulb etc. Here is the sample code for that.

Fan.java

```
package structural.bridge;
/**
```

```
* Implement the switch for Fan
*/
public class Fan implements Switch {
        // Two positions of switch.
        public void switchOn() {
        System.out.println("FAN Switched ON");
        }

        public void switchOff() {
        System.out.println("FAN Switched OFF");
        }
}// End of class
```

And implementation as Bulb.

Bulb.java

```
package structural.bridge;
/**
* Implement the switch for Fan
*/
public class Bulb implements Switch {
        // Two positions of switch.
        public void switchOn() {
        System.out.println("BULB Switched ON");
        }

        public void switchOff() {
        System.out.println("BULB Switched OFF");
        }
}// End of class
```

```
class Node {
    public int  value;
    public Node prev, next;
    public Node( int i ) { value = i; }
}

class Stack {
    private StackImpl impl;
    public Stack( String s ) {
        if      (s.equals("array")) impl = new StackArray();
        else if (s.equals("list"))  impl = new StackList();
        else System.out.println( "Stack: unknown parameter" );
    }
    public Stack()                { this( "array" ); }
    public void    push( int in ) { impl.push( in ); }
    public int     pop()          { return impl.pop(); }
    public int     top()          { return impl.top(); }
    public boolean isEmpty()      { return impl.isEmpty(); }
```

```
    public boolean isFull()       { return impl.isFull(); }
}

class StackHanoi extends Stack {
    private int totalRejected = 0;
    public StackHanoi()           { super( "array" ); }
    public StackHanoi( String s ) { super( s ); }
    public int reportRejected()   { return totalRejected; }
    public void push( int in ) {
        if ( ! isEmpty()  &&  in > top()) {
            totalRejected++;
        }
        else {
            super.push( in );
        }
    }
}

class StackFIFO extends Stack {
    private StackImpl temp = new StackList();
    public StackFIFO() {
        super( "array" );
    }
    public StackFIFO( String s ) {
        super( s );
    }
    public int pop() {
        while ( ! isEmpty()) {
            temp.push( super.pop() );
        }
        int ret =  temp.pop();
        while ( ! temp.isEmpty()) {
            push( temp.pop() );
        }
        return ret;
    }
}

interface StackImpl {
    void    push( int i );
    int     pop();
    int     top();
    boolean isEmpty();
    boolean isFull();
}

class StackArray implements StackImpl {
    private int[] items = new int[12];
    private int   total = -1;
    public void push( int i ) {
        if ( ! isFull()) {
            items[++total] = i;
        }
        public boolean isEmpty() {
          return total == -1;
        }
        public boolean isFull() {
```

```
          return total == 11;
        }
        public int top() {
            if (isEmpty()) return -1;
            return items[total];
        }
        public int pop() {
            if (isEmpty()) return -1;
            return items[total--];
        }
    }
}

class StackList implements StackImpl {
    private Node last;
    public void push( int i ) {
        if (last == null)
            last = new Node( i );
        else {
            last.next = new Node( i );
            last.next.prev = last;
            last = last.next;
        }
    }
    public boolean isEmpty() {
        return last == null;
    }
    public boolean isFull() {
        return false;
    }
    public int top() {
        if (isEmpty()) return -1;
        return last.value;
    }
    public int pop() {
        if (isEmpty()) return -1;
        int ret = last.value;
        last = last.prev;
        return ret;
    }
}

class BridgeDisc {
    public static void main( String[] args ) {
        Stack[] stacks = { new Stack( "array" ), new Stack( "list" ),
                           new StackFIFO(), new StackHanoi() };
        for (int i=1, num; i < 15; i++) {
            for (int j=0; j < 3; j++) {
                stacks[j].push( i );
            }
        }
        java.util.Random rn = new java.util.Random();
        for (int i=1, num; i < 15; i++) {
            stacks[3].push( rn.nextInt(20) );
        }
        for (int i=0, num; i < stacks.length; i++) {
            while ( ! stacks[i].isEmpty()) {
```

```
                System.out.print( stacks[i].pop() + "  " );
            }
            System.out.println();
        }
        System.out.println("total rejected is " +
((StackHanoi)stacks[3]).reportRejected() );
    }
}
```

Rules of thumb

- Adapter makes things work after they're designed; Bridge makes them work before they are.
- Bridge is designed up-front to let the abstraction and the implementation vary independently. Adapter is retrofitted to make unrelated classes work together.
- State, Strategy, Bridge (and to some degree Adapter) have similar solution structures. They all share elements of the "handle/body" idiom. They differ in intent - that is, they solve different problems.
- The structure of State and Bridge are identical (except that Bridge admits hierarchies of envelope classes, whereas State allows only one). The two patterns use the same structure to solve different problems: State allows an object's behavior to change along with its state, while Bridge's intent is to decouple an abstraction from its implementation so that the two can vary independently.
- If interface classes delegate the creation of their implementation classes (instead of creating/coupling themselves directly), then the design usually uses the Abstract Factory pattern to create the implementation objects.